

氣泡排序法

```
void BubbleSort(int A[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (A[j] > A[j + 1]) {
                temp = A[j];           // 交換 A[j] 和 A[j + 1]
                A[j] = A[j + 1];
                A[j + 1] = temp;
            }
        } //內圈的 FOR
    } //外圈的 FOR
}
```

快速排序法

```
void Qsort(int A[], int L, int R) {
    int i, j, P;
    if (L < R) {
        i = L;
        j = R;
        P = A[L]; // PK 值
        While (i < j) {
            while (i < R && A[i] < P) {i++;} // i 由右往左找(->)，比 PK 大的值
            while (j > L && A[j] > P) {j--;} // j 由左往右找(<-)，比 PK 小的值
            if (i < j) {SWAP(A[i], A[j]);} // 如果 A[i] 小於 A[j]，則交換兩個元素
        }
        SWAP(A[L], A[j]); //j 超過 i 時 A[j] 和 PK 交換位置
        Qsort(A, L, j - 1);
        Qsort(A, j + 1, R);
    }
}
```

Fabonacci 序列

遞迴版本	非遞迴版本
<pre>int fib(int n) { if(n >= 1) {return n;} return (fib(n - 1) + fib(n - 2)); }</pre>	<pre>int fib(int N){ if(N >= 1) {return N;} int a = 0, b = 1, c = 0; for(int i = 2; i <= N){ c = a + b; a = b; b = c; } return c; }</pre>

最大公因數

遞迴版本	非遞迴版本
<pre>int GCD(int num1,int num2) { if(num2==0){return num1;} return GCD(num2,num1%num2); }</pre>	<pre>int gcd(int m,int n) { int t,r; if (m<n){ t=m; m=n;n=t; } //debug 用 //確保大數除小數 while((m%n)!=0) { //輾轉相除 r=m%n; m=n; n=r; } return n; }</pre>

二元搜尋法

//二元搜尋法，需先排序過才行

遞迴版本	非遞迴版本
<pre>int BS(int data[],int low,int high,int key) { if (low > high) {return -1;} //找不到值 int mid = (low + high) / 2; if (key == data[mid]) { return mid;}//找到值 else if (key > data[mid]) { low = mid + 1;} else { high = mid - 1;} return BS(data, low, high, key); }</pre> 1、確認是否在序列中，low > high 找不到值 2、取中間值 (low + high) / 2 3、狀況 1：找到值 狀況 2：KEY 比中間值大，往大找，重設下限 low 狀況 3：KEY 比中間值小，往小找，重設上限 high 4、跑遞迴	<pre>int BS(int data[], int size, int key) { int low = 0; int high = size - 1; while (low <= high) { int mid = (low + high) / 2; if(key == data[mid]) {return mid;}//找到值 else if (key > data[mid]) {low = mid + 1;} else {high = mid - 1;} } return -1 ; //找不到值 }</pre> 1、設定上/下限 low、high 2、跑 while 迴圈，條件在序列中 low <= high 3、取中間值 (low + high) / 2 4、狀況 1：找到值 狀況 2：KEY 比中間值大，往大找，重設下限 low 狀況 3：KEY 比中間值小，往小找，重設上限 high

選擇排序法 - Selection Sort

```
void selection(int arr[], int len){
    for(int i = 0; i < len; ++i){
        int index = i;
        for(int j = i + 1; j < len; ++j){
            if(arr[index] > arr[j]) // 如果再 arr 後面有找到更小的數，把它記錄下來
                index = j;
        }
        if(index != i){ swap(arr[index], arr[i]);} // 最後在做交換的動作
    }
}
```

插入排序法 - Insertion Sort

```
void insertion_sort(int arr[], int len) {
    for (int i = 1; i < len; i++) {
        int temp = arr[i]; // 當前待插入的元素
        int j = i;
        while (j > 0 && arr[j - 1] > temp) { // 向右移動大於 temp 的元素
            arr[j] = arr[j - 1];
            j--;
        }
        arr[j] = temp; // 插入元素到正確位置
    }
}
```

```

// Link List
#include <stdio.h>
#include <stdlib.h> // malloc 函示需要載入這個標題檔

struct node {
    int data;
    struct node * next;
};
typedef struct node NODE; // 定義 NODE 關鍵字代表 node 結構體
typedef struct node * NODEp; // 定義 NODEp 關鍵字代表指向 node 結構體的 pointer

int main(void) {
    int arr[] = {1, 2, 3}; // 每個節點的資料
    NODEp first; // 要指向第一個節點的 pointer
    NODEp current, prev; // 定義兩個 pointer 用來追蹤節點
    for (int i = 0; i < 3; i++) {
        current = (NODE *) malloc(sizeof(NODE)); // 建立一個 NODE 類型大小的空間，並讓 current 指向
        current -> data = arr[i]; // 儲存資料
        // 如果是第一個節點，就讓 first 指向該節點
        if (i == 0) {first = current; }
        else {prev -> next = current; }
        current -> next = NULL;
        prev = current;
    }
    NODEp p = first;
    while (p != NULL) {
        printf("%d ", p -> data);
        p = p -> next;
    }
    printf("\n");
    NODEp pp = first;
    while (pp != NULL) {
        first = first -> next; // 先將 first 指向下一個節點，不然之後會錯誤
        free(pp); // 釋放該節點的記憶體
        pp = first;
    }
    return 0;
}

```

```

//二元樹 (Pre-order 追蹤)
#include <stdio.h>
#include <stdlib.h>

struct TreeNode { // 定義節點
    int val;
    struct TreeNode * left;
    struct TreeNode * right;
};

struct TreeNode * createNode(int val) // 新增節點
{
    struct TreeNode * newNode = (struct TreeNode * ) malloc(sizeof(struct TreeNode));
    newNode -> val = val;
    newNode -> left = NULL;
    newNode -> right = NULL;
    return newNode;
}

void preorderTraversal(struct TreeNode * root) // Preorder 追蹤
{
    if (root == NULL) // if the current node is empty
        return;
    printf("%d ", root -> val); // 拜訪節點
    preorderTraversal(root -> left); // 追蹤左子樹
    preorderTraversal(root -> right); // 追蹤右子樹
}

int main() {
    // creating example tree
    struct TreeNode * root = createNode(1);
    root -> left = createNode(2);
    root -> right = createNode(3);
    root -> left -> left = createNode(4);
    root -> left -> right = createNode(5);
    printf("Preorder Traversal: ");
    preorderTraversal(root);
    return 0;
}

```

封裝 (Encapsulation) 將物件的狀態和行為封裝在一個單元中， 並且只對外部提供有限的接口來訪問物件， 從而保護物件的內部 狀態不被直接訪問或修改。

```
public class Car {
    private String make;
    private String model;
    private int year;
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
    public void drive() {
        System.out.println("The " + this.make + " " + this.model + " is now being driven");
    }
}
```

繼承 (Inheritance) 通過建立一個新物件， 讓其繼承一個或多個已有物件的屬性和方法， 從而使得新物件可以重複使用已有物件的程式碼， 同時也可以擴展或修改已有物件的功能 **多型 (Polymorphism)** 讓物件可以根據上下文表現出不同的行為， 即一個物件可以被當作多個不同的物件使用， 從而提高程式的 靈活性和可擴展性。 多型通常可以通過多載 (Overloading) 和複寫 (Overriding) 來實現。

```
public class SportsCar extends Car {
    public SportsCar(String make, String model, int year) {
        super(make, model, year);
    }

    @Override
    public void drive() {
        System.out.println("The " + this.make + " " + this.model + " is now being driven fast");
    }

    public void drive(int speed) {
        System.out.println("The " + this.make + " " + this.model + " is now being driven at " +
            speed + " mph");
    }
}
```

多載 (Overloading): SportsCar 子類別中定義了另一個 drive 方法， 但這次我們添加了一個額外的 speed 參數。 這就是方法重載的體現。 當我們調用帶有 speed 參數的 drive 方法時， Java 會自動識別我們調用的是哪一個方法， 並且根據參數的個數和類型來決定使用哪一個方法。

複寫 (Overriding): SportsCar 繼承了 Car 父類別， 並且重寫了父類別中的 drive 方法。 在 SportsCar 子類別中的 drive 方法上， 使用了 @Override 注解， 這就是方法重寫的體現。 當我們創建 SportsCar 類的對象時， 調用的就是子類別中的 drive 方法， 而不是父類別中的 drive 方法。